

FSCENECHARGER appliqué à la scène initiale (ID=0)

Trace complète de l'exécution de `F_SCENE_CHARGER` avec `B=0`, tel qu'appelé depuis `F_Z80LIB_LANCER`
(`LD B,0 : CALL F_SCENE_CHARGER`).

Rappel du code

F_SCENE_CHARGER

```
LD    A, IM1_NOT_READY
LD    (IM1_INDEX), A           ; Invalide le vecteur IM1
LD    A, B                     ; A = ID_SCENE_xxx (0..N-1 attendu)
CP    SCENE_TABLE_COUNT
RET   NC                       ; ID hors limites -> inconnu, sortie
```

```
ADD   A, A                     ; A = ID*2
LD    E, A
ADD   A, A                     ; A = ID*4
ADD   A, E                     ; A = ID*4 + ID*2 = ID*6
LD    L, A
LD    H, 0
LD    DE, SCENE_TABLE
ADD   HL, DE                   ; HL = SCENE_TABLE + ID*6
```

.scene_found

```
LD    E, (HL)
INC   HL
LD    D, (HL)                 ; DE = adresse F_SCENE_xxx
INC   HL
PUSH  DE
LD    E, (HL)
INC   HL
LD    D, (HL)                 ; DE = adresse ARRAY_IM1_xxx
INC   HL
LD    (IM1_CURRENT), DE
LD    E, (HL)
INC   HL
LD    D, (HL)                 ; DE = adresse F_SCENE_xxx_INITIALISATION
POP   HL
LD    (SCENE_ACTUEL+1), HL
PUSH  DE
RET
```

Trace pas à pas avec B=0

Étape	Instruction	Résultat
Invalidation	LD A, IM1_NOT_READY puis LD (IM1_INDEX), A	IM1_INDEX bloqué, l'ISR ne dispatche plus rien tant qu'un vrai VBL ne survient pas
Chargement ID	LD A, B (B=0)	A = 0
Vérification bornes	CP SCENE_TABLE_COUNT puis RET NC	0 < SCENE_TABLE_COUNT (forcément vrai si au moins une scène existe) -> pas de sortie anticipée
Calcul offset	ADD A, A (x2) -> LD E, A -> ADD A, A (x4) -> ADD A, E (+x2)	A = 0*6 = 0 -- offset nul, la première entrée de la table
Adresse finale	LD HL, SCENE_TABLE puis ADD HL, DE	HL = SCENE_TABLE + 0 = SCENE_TABLE -- pointe directement sur la première entrée, sans aucun décalage

Avec ID=0, le calcul d'offset produit 0 — c'est le cas qui aurait masqué le bug x8 découvert lors de la mise au point de cette routine (0*8=0 donne la même adresse que 0*6=0), d'où le fait que ce bug ne s'est révélé qu'en testant un ID>=1.

Lecture de la première entrée de SCENE_TABLE

En supposant que la scène ID=0 est le menu (convention classique) :

```
SCENE_TABLE
    DW F_SCENE_MENU, ARRAY_IM1_MENU, F_SCENE_MENU_INIT    ; <- entrée lue ici
    DW F_SCENE_JEU,  ARRAY_IM1_JEU,  F_SCENE_JEU_INIT
    ...
```

Lecture	Registre	Valeur récupérée
1er DW	DE puis PUSH	adresse de F_SCENE_MENU (la logique de la scène, appelée à 50Hz)
2e DW	DE -> IM1_CURRENT	adresse de ARRAY_IM1_MENU (table des 6 sous-tâches IM1 pour cette scène)
3e DW	DE	adresse de F_SCENE_MENU_INIT (routine d'initialisation, exécutée une seule fois)

Le saut final

```
POP  HL                ; HL = F_SCENE_MENU (le 1er DW, remis en jeu)
LD   (SCENE_ACTUEL+1), HL ; réécrit l'opérande du CALL dans BOUCLE_PRINCIPALE
PUSH DE                ; DE = F_SCENE_MENU_INIT (le 3e DW)
RET                                ; dépile DE et saute dessus -> exécute l'init
```

Après ce `RET`, l'exécution reprend directement dans `F_SCENE_MENU_INIT` — pas de retour à `F_Z80LIB_LANCER` avant que cette initialisation soit terminée. C'est `F_SCENE_MENU_INIT` elle-même qui devra faire son propre `RET` final pour revenir dans le flot normal (retour à `F_Z80LIB_LANCER`, juste après le `CALL F_SCENE_CHARGER`).

Récapitulatif de l'état final après ID=0

Variable	Valeur
<code>IM1_INDEX</code>	<code>IM1_NOT_READY</code> (jusqu'au prochain vrai VBL)
<code>IM1_CURRENT</code>	<code>ARRAY_IM1_MENU</code>
<code>SCENE_ACTUEL+1</code>	<code>F_SCENE_MENU</code> (opérande du <code>CALL</code> dans <code>BOUCLE_PRINCIPALE</code>)
Exécution en cours	<code>F_SCENE_MENU_INIT</code>

Une fois `F_SCENE_MENU_INIT` terminée et son `RET` exécuté, le programme revient dans `F_Z80LIB_LANCER`, juste après le `CALL F_SCENE_CHARGER` — la ligne suivante étant `LD HL, FRAME_READY`, qui prépare l'entrée dans `BOUCLE_PRINCIPALE`.